

SE 352/452 - Object-Oriented Enterprise Computing

Final Project: Full-Stack Spring Boot Application: “Gearwood Puzzle Store”

100 pts (25% of course grade)

Gearwood Puzzle Store Application Requirements

1. Application Overview

Students will develop a complete Spring Boot MVC e-commerce application named **Gearwood Puzzle Store**. The application represents an online retailer specializing in wooden mechanical puzzles and model kits.

The system must provide both customer-facing shopping functionality and administrator-only management functionality. The project should simulate the structure and behavior of a realistic modern e-commerce web application while remaining appropriately scoped for an academic software engineering project. Note that AI tool assistance is allowed.

The application must allow users to:

- Browse and search products
- View detailed product information
- Register and authenticate accounts
- Manage a shopping cart
- Complete the checkout process
- View order history and order details
- Manage products through administrator-only functionality

The completed project should demonstrate:

- Proper use of Spring Boot and Spring MVC
- Layered application architecture
- Use of Spring Security authentication and authorization
- Persistent relational data storage
- Clean separation of concerns
- Professional user interface design
- Proper validation and error handling
- Secure access control

The application should feel cohesive and professionally designed rather than appearing as a collection of disconnected pages or features. Students are encouraged to focus on usability, consistency, and maintainability throughout the implementation.

2. Product Catalog Requirements

2.1 Product Listing

The application must display a list of available products. Only products marked as active may appear in customer-facing product listings.

Each product listing must display:

- Primary product image
- Product name
- Manufacturer
- Category
- Difficulty
- Price
- Short description

Products on the listing page must be displayed using a card-style layout.

Clicking a product card must navigate the user to the product details page.

You are encouraged to create a clean and visually organized browsing experience.

2.2 Product Search and Filtering

The application must support flexible product searching and filtering to help users efficiently browse the catalog.

Users should be able to narrow product results using one or more filtering criteria simultaneously. The filtering system should behave similarly to modern online retail websites where users can progressively refine search results.

The application must support searching and filtering products using one or more of the following criteria:

- Keyword (case-insensitive) – searches in product's name
- Category
- Difficulty
- Minimum price
- Maximum price

Users must be able to combine multiple filtering criteria within a single request. If no filters are supplied, all active products should be displayed.

The product listing page must display the number of products currently shown after filtering is applied.

Example:

- Products (30)
- Products (8)

The displayed count should update dynamically as filtering criteria change.

2.3 Product Details

The application must provide a product details page containing complete information for a selected product.

The page must display:

- Product images
- Product name
- Manufacturer
- Number of pieces
- Difficulty
- Category
- Price (BigDecimal)
- Short description (String)
- Long description (String)

The layout should organize information clearly to help users evaluate products before purchase.

2.4 Product Images

Each product must support multiple product images.

Images should be loaded dynamically from a project-located image folder. Images are to be named after the product code. For example, product code *P1008* will have images *P1008-01.jpg*, *P1008-02.jpg*, *P1008-03.jpg*, etc. The “*-01.jpg” images will be the primary product image.

Students are not required to implement image upload functionality. Images may be stored directly in the project image directory (static/images).

The product details page must support:

- One larger primary image
- Multiple selectable thumbnail images

When a thumbnail image is selected, the primary image should update to display the selected image.

This functionality should allow users to browse multiple views of a product.

2.5 Product Data

Each product must contain the following information:

- Product code (String), unique across all products, admin provided
- Name (String)
- Manufacturer (a reference to a Manufacturer object)
- Number of pieces (Integer)
- Difficulty (one of EASY, MEDIUM, HARD)
- Category (one of MECHANICAL, HISTORICAL, NAUTICAL, SCI_FI, FANTASY, ARCHITECTURE, DECORATIVE)
- Price (BigDecimal)
- Short description (String)
- Long description (String)
- Active/inactive status (boolean)

Each Manufacturer must contain:

- Name (String)
- Address (String)
- Contact email (String)
- Contact phone (String)

Each User must contain:

- First name (String)
 - Last name (String)
 - Email (String), unique across all users
 - Password (String), 6 character minimum
 - Role (a reference to a Role object)
-

3. User Account Requirements

3.1 Authentication and Account Management

The application must allow users to:

- Register for an account
- Log in
- Log out
- View an account page
- Edit profile information (First/Last Name, *email is not editable*)
- Change password

Users must only be able to access and modify their own account information.

3.2 Order History Access

Authenticated users must be able to:

- View order history
- View detailed information for previous orders

Users may only access their own orders.

4. Shopping Cart Requirements

4.1 Cart Functionality

The shopping cart serves as a temporary holding area for products a customer intends to purchase.

Authenticated users must be able to:

- Add products to the cart
- Specify product quantity
- View cart contents
- Update item quantities
- Remove items from the cart

The shopping cart must persist for the authenticated user across requests.

The cart must only be cleared after successful checkout.

4.2 Login Redirect Behavior

If a user attempts to add a product to the cart before logging in:

1. The user must be redirected to the login page.
2. After successful authentication, the original action must continue automatically.

4.3 Product Details Cart Behavior

The product details page should display different cart interaction behavior depending on the user's authentication state.

For unauthenticated users:

The UI should:

- Display a message indicating that login is required before adding items to the cart
- Display a login link or button

For authenticated users:

The UI must:

- Allow quantity selection
- Allow products to be added directly to the cart

4.4 Cart Summary

The cart page must display an order summary containing:

- Subtotal
- Estimated shipping
- Estimated tax
- Estimated total

If free shipping is applied, the UI should clearly indicate that free shipping has been granted.

The order summary should update automatically when cart contents change.

5. Checkout Requirements

5.1 Checkout Information

Authenticated users must be able to complete checkout by entering:

- Shipping name
 - Street address
 - City
 - State
 - ZIP code
 - Payment information (*cardholder name, card number, expiration, CVV*)
-

5.2 Checkout Processing

The checkout process should simulate the behavior of a realistic online purchasing workflow.

The system should guide the user through reviewing the cart, entering shipping and payment information, validating the order, processing payment, and generating an order confirmation.

The checkout process must:

- Calculate subtotal
 - Calculate tax using a consistent tax rule (Use tax of 8.25%)
 - Calculate shipping using a defined rule (Free shipping on orders of \$75.00 or more)
 - Apply free shipping when applicable
 - Process payment using a dummy payment processor (provided)
 - Create an order when payment is approved
 - Reject checkout when payment is declined
 - Clear the cart after successful checkout
-

5.3 Checkout Validation

Checkout must fail if:

- The cart is empty
- Any product in the cart is inactive
- Required checkout information is missing
- Payment is declined

The application must display clear user-friendly error messages when checkout fails.

5.4 Checkout Success

After successful checkout, the user must be redirected to a confirmation page.

The confirmation page must display:

- A success message
- The generated order number
- Navigation options:
 - Continue shopping
 - View account/order history

The displayed order number must be an application-defined value and must not expose the database ID.

6. Order Requirements

6.1 Order Data

The application must create and store orders containing:

- Order number (application-defined String)
- User (a reference to a User object)
- Order date/time (a LocalDateTime object)
- Order status (one of PAID, CANCELLED)
- Total amount (BigDecimal)
- Transaction ID (String)
- Shipping information (name, address, address2, city, state, zip)
- Ordered items (a collection of OrderItem objects)

Each order must include an order status, as indicated above. For this project, a successful checkout should create an order with status PAID. Declined payments should not create an order; instead, the user should remain on the checkout page and see a clear error message. The CANCELLED status should be present but is not necessary or needed for this project. Assume it's there as a future TBD behavior.

Orders should preserve the financial breakdown used at checkout, including subtotal, tax amount, shipping amount, and final total, so that order details remain historically accurate.

6.2 Order History Data

Each order item must preserve historical purchase information, including:

- Order reference (a reference to an Order object)

- Product reference (a reference to an Product object)
- Product name at time of purchase (String)
- Unit price at time of purchase (BigDecimal)
- Quantity (Integer)
- Line total (BigDecimal)

Orders must preserve historical accuracy even if product information changes later.

6.3 Order History Display

The Account page must contain a brief order summary and include navigation to a dedicated Order History page.

Each order summary entry must display:

- Order number
- Order date
- Order status
- Order total
- A link or button to view order details

The order history should be presented in a clean and readable format.

6.4 Order Details Page

The application must provide a dedicated order details page that allows users to review the contents and status of a previously completed purchase.

The page should present information in a clean and organized layout that resembles a professional e-commerce order receipt or invoice.

The application must provide an order details page displaying:

- Order number
- Order date/time
- Order status
- Transaction ID
- Shipping information
- Order total

The ordered items section must display:

- Product image
- Product name
- Quantity
- Unit price at time of purchase
- Line total

The page should allow users to review completed purchases in a clear and organized manner.

7. Administrator Product Management Requirements

7.1 Administrative Access

Only administrator users may access product management functionality.

Admin controls appear inside the regular product pages when an admin is logged in.

Administrator functionality must be protected using role-based authorization.

7.2 Product Management Features

Administrators are responsible for maintaining the store catalog and controlling which products are available to customers.

The administrator functionality should provide a clear management workflow for maintaining product data without directly modifying the database.

Administrators must be able to:

- Create products
- Edit products
- View active/inactive product status
- Activate products
- Deactivate products

Administrators must be able to easily access product creation functionality from the product details page.

7.3 Administrative Product Views

Administrative product views must:

- Match customer (non-admin) view.
- Display both *active* and *inactive* products
- *Clearly indicate product status*

The application must conditionally displaying administrative controls within the standard product listing and product details pages when an administrator is logged in.

Administrator actions are:

- Create product
- Edit product
- Activate/Deactivate product

7.4 Product Creation/Edit Rules

Products must be created by an admin with the following data content:

- | | |
|----------------|--|
| • Product code | • Category |
| • Name | • Price |
| • Manufacturer | • Short Description |
| • Piece count | • Long Description |
| • Difficulty | • Acquired Date (<i>should be set to current date</i>) |

Images should be loaded dynamically from a project-located image folder. Images are to be named after the product code. For example, product code *P1008* will have images *P1008-01.jpg*, *P1008-02.jpg*, *P1008-03.jpg*, etc. The “*-01.jpg” images will be the primary product image.

Students are not required to implement image upload functionality. Images may be stored directly in the project image directory (static/images).

7.5 Product Activation/Deactivation Rules

Products must be marked as active/inactive within the product details page. Products can be made active or inactive at any time by an admin user. Inactive products cannot be seen by non-admin users in the product display.

7.6 Product Deletion Rules

Products must not be permanently deleted if they are associated with:

- Existing orders
- Shopping carts

Such products should instead be marked inactive and cannot be deleted.

Products not associated with existing carts or orders may be permanently deleted. The related images must also be deleted from the static/images folder.

8. Active and Inactive Product Requirements

8.1 Customer-Facing Behavior

Only active products may appear in customer-facing product listings, search results, and product detail pages. *(Administrators may still see and access inactive products for management purposes.)*

Inactive products:

- Must not appear in normal customer browsing
- Must not be addable to the cart
- Must not be purchasable

If a user attempts to access an inactive product directly through a URL, the application must prevent access through:

- Redirect behavior
 - An error page
 - Another appropriate mechanism
-

8.2 Administrator Behavior

Administrators must be able to:

- View inactive products
 - Edit inactive products
 - Reactivate inactive products
-

9. Security Requirements

9.1 Spring Security

The application must use Spring Security.

The application must support two security roles:

- CUSTOMER
 - ADMIN
-

9.2 Customer Permissions

Customer users must be able to:

- Browse products
 - Manage the shopping cart
 - Complete checkout
 - View their own account information
 - View their own order history and order details
-

9.3 Administrator Permissions

Administrator users must be able to:

- Manage products
 - Access administrator-only functionality
-

9.4 Authorization Rules

The application must properly enforce authorization rules to ensure that users can only access functionality appropriate for their role.

Security restrictions must be enforced both in the user interface and at the server/controller level. Simply hiding buttons or navigation links is not sufficient protection.

The application must enforce security restrictions both:

- In the user interface
- At the controller/server level

Examples include:

- Unauthenticated users should not see account or logout links
- Non-admin users should not see administrator controls
- Users must not access another user's orders or account information
- Administrator-only routes must be protected

The application navigation should adapt dynamically based on authentication state and user role.

10. Data Requirements

The application must include seeded sample data for:

- Products
- Manufacturers (*Data-seeded only, no add/edit/remove through the website*)
- Users/Roles

The raw data for manufacturers and products (including project images) will be provided.

At least one administrator account must be preconfigured: `admin.user@gmail.com`

Product data must be sufficient to demonstrate:

- Product searching and filtering
 - Product details functionality
 - Cart and checkout functionality
 - Administrator product management
-

11. Technical Requirements

11.1 Required Technologies

The application must use:

- Spring Boot
 - Spring MVC
 - Thymeleaf
 - Spring Security
 - Spring Data JPA
 - MySQL
-

11.2 Application Architecture

The project must follow a clean layered architecture that separates presentation logic, business logic, and data access responsibilities.

Students should avoid placing all functionality directly inside controllers or entity classes. Responsibilities should be clearly divided among the different application layers.

The application must follow a layered architecture containing:

- Controllers (web layer)
- Services (business logic layer)
- Repositories (data access layer)
- Entities/models

Business logic must be implemented in the service layer rather than directly in controllers.

11.3 Validation and Error Messaging

The application must display clear and user-friendly validation or error messages when operations fail.

Examples include:

- Invalid login credentials
- Duplicate account registration
- Invalid form input
- Declined payment
- Empty required fields
- Unauthorized access attempts

Error messages should help users understand how to correct the problem.

12. User Interface Requirements

12.1 Required Pages

The application must include Thymeleaf pages for:

- Home page
- Product list page
- Product details page
- Product create/edit page
- Login page
- Registration page
- Cart page
- Checkout page
- Checkout success page
- Account page
- Edit account page
- Change password page
- Order history page

12.2 Shared Navigation

The application must include a consistent shared navigation/header area across major pages.

The navigation must visually appear consistent throughout the application.

For unauthenticated users:

The navigation should display links such as:

- Home
- Products/Search
- Login
- Register

For authenticated users:

The navigation should display links such as:

- Home
- Products/Search
- Cart
- Account
- Logout

The application should display a short welcome message containing the user's first name.

12.3 Styling Consistency

The application must use shared CSS resources and reusable layout structures to provide a consistent appearance across all pages.

The overall user interface should appear consistent, visually cohesive and professionally designed. Pages should not appear disconnected or use conflicting layouts and styling patterns.

The application must use shared CSS resources to provide consistent styling across all pages.

Consistency should include:

- Navigation/header appearance
- Buttons
- Forms
- Cards and panels
- Typography
- Spacing and alignment

Students are encouraged to create a professional and visually organized user experience.

The exact visual appearance does not need to perfectly match the example screenshots.

12.4 Additional UI Behavior

The application must support the following behavior:

- Clicking a product card navigates to the product details page
- Product images support dynamic image switching
- Users are redirected back to intended actions after login when appropriate

13. Error Handling Requirements

The application must handle errors gracefully. Examples include:

- Invalid login attempts
- Invalid registration data
- Empty checkout form fields
- Declined payments
- Empty cart checkout attempts
- Inactive products
- Attempts to add inactive products to the cart
- Unauthorized access attempts
- Invalid or missing request data

The application should display clear, user-friendly feedback whenever an error occurs.

The application should avoid exposing raw exception messages or stack traces to end users.

14. Project Grading

This project is a 100 point project, and comprises 25% of the course grade. Project grades are the sum of points earned on the following sections. Note the *Basic Project Requirements* are mandatory. All the remaining elements below (in italics) are optional. You may do any number of them, including all of them. Note that *Order Details* requires *Order History* to be completed. The *Product Search/Filtering*, *Order History*, and *Product Delete* are independent of each other.

Basic Project Requirements (in black):	60%
<i>Product Search/Filtering, section 2.2 (in red):</i>	10%
<i>Order History, section 3.2, 6.2, 6.3 (in blue):</i>	10%
<i>Order Details, section 6.4 (in green):</i>	10%
<i>Product Create/Edit, section 7.4 (in brown):</i>	10%
<i>Product Delete, section 7.6 (in purple):</i>	10%
Maximum Score:	110%

Each above section will be graded, the sum of the grades will be your project grade. Any section not implemented will not receive a grade.

Grading on the project sections listed will be based upon the following:

- Correct use of Thymeleaf syntax to display dynamic data in all HTML templates.
- Proper implementation all controllers, services, repositories, entities, beans, etc.
- Proper handling of error situations.
- Pages reasonably match capabilities shown in the provided screenshot examples in functionality and content. Styling is clean, complete, and readable.
- All required navigation links (links between pages) are present and function correctly.
- Code is clean, well-structured, follows best practices, and adheres to provided project structure.
- The application works as intended with no broken functionality or critical errors.
- Submissions are only accepted via D2L.
- NO LATE SUBMISSIONS ARE ALLOWED

○ **Project Assistance:**

- If you are stuck on some design or code related problem *that you have exhaustively researched and/or debugged yourself*, you can send me a ZIP file of your entire project (following the instructions found on D2L) so that I can examine the problem. *Be sure to follow the instructions found on D2L (Content → Emailing IntelliJ Projects to the Instructor) when doing so.*
- All emailed assistance requests must include a detailed description of the problem, and the details of what steps you have already taken in trying to determine the source of the problem.

• **Submissions:**

- When submitting, you should submit a ZIP file of your entire project folder, as described on D2L at *Content → Zipping IntelliJ Projects for Submission*.
- Submissions must reflect the concepts and practices cover in class, and the requirements specified in this document.
- Please review the Academic Integrity and Plagiarism section of the syllabus before, during, and after working on this assignment. All work must be your own. AI tool assistance is allowed.
- When submitting, you should submit a ZIP file of your entire project folder, as described on D2L at *Content → Zipping IntelliJ Projects for Submission*.